IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

# Yenepoya Institute of Arts, Science, Commerce & Management.
# Project- High Level Design
# On
# Identity Security Framework for Retail

**Student Name:**

**Liyakath Sulfeekar**

**23BCCE110**

# Table of Contents

**1. Introduction** ---------- 1
1.1. Scope of the Document ---------- 1
1.2. Intended Audience ---------- 1
1.3. System Overview ---------- 1
**2. System Design** ---------- 2
2.1. Application Design ---------- 2
2.2. Process Flow ---------- 3
2.3. Information Flow ---------- 5
2.4. Components Design ---------- 6
2.5. Key Design Considerations ---------- 7
2.6. API Catalogue ---------- 8
**3. Data Design** ---------- 9
3.1. Data Model ---------- 9
3.2. Data Access Mechanism ---------- 10
3.3. Data Retention Policies ---------- 11
3.4. Data Migration ---------- 11
**4. Interfaces** ---------- 11
**5. State and Session Management** ---------- 12
**6. Caching** ---------- 13
**7. Non-Functional Requirements** ---------- 14
7.1. Security Aspects ---------- 14
7.2. Performance Aspects ---------- 14
**8. Reference** ---------- 15

# 1. INTRODUCTION

The Identity Security Framework for Retail is a web-based platform designed to provide secure authentication and role-based access control for a retail business operating both online and in-store. As retail businesses increasingly manage sensitive customer data, sales records, and staff information digitally, ensuring that only verified users can access appropriate resources has become a critical security requirement.

The platform, branded as Retail Shoppeee — The Complete Retail Shop, implements layered identity security for three user roles: Admin, Staff, and Customer. Each user is authenticated through a two-factor process involving password hashing and email-based One-Time Password (OTP) verification, and is then directed to a role-specific dashboard.

## 1.1. Scope of the Document

This document covers the high-level architecture, component interactions, process and data flow diagrams, data design, interface overview, session management strategy, and non-functional requirements. Detailed implementation specifications are addressed in the Low-Level Design (LLD) document.

## 1.2. Intended Audience

This document is intended for the academic evaluators and faculty, the IBM industry mentor, the internal project guide (Munisah), and developers who may extend or maintain the system.

## 1.3. System Overview

The system consists of a web-based retail management platform where users interact through a browser-based interface. The Flask backend handles authentication, authorisation, and database management. A security layer validates each user and enforces role-specific access across all modules.

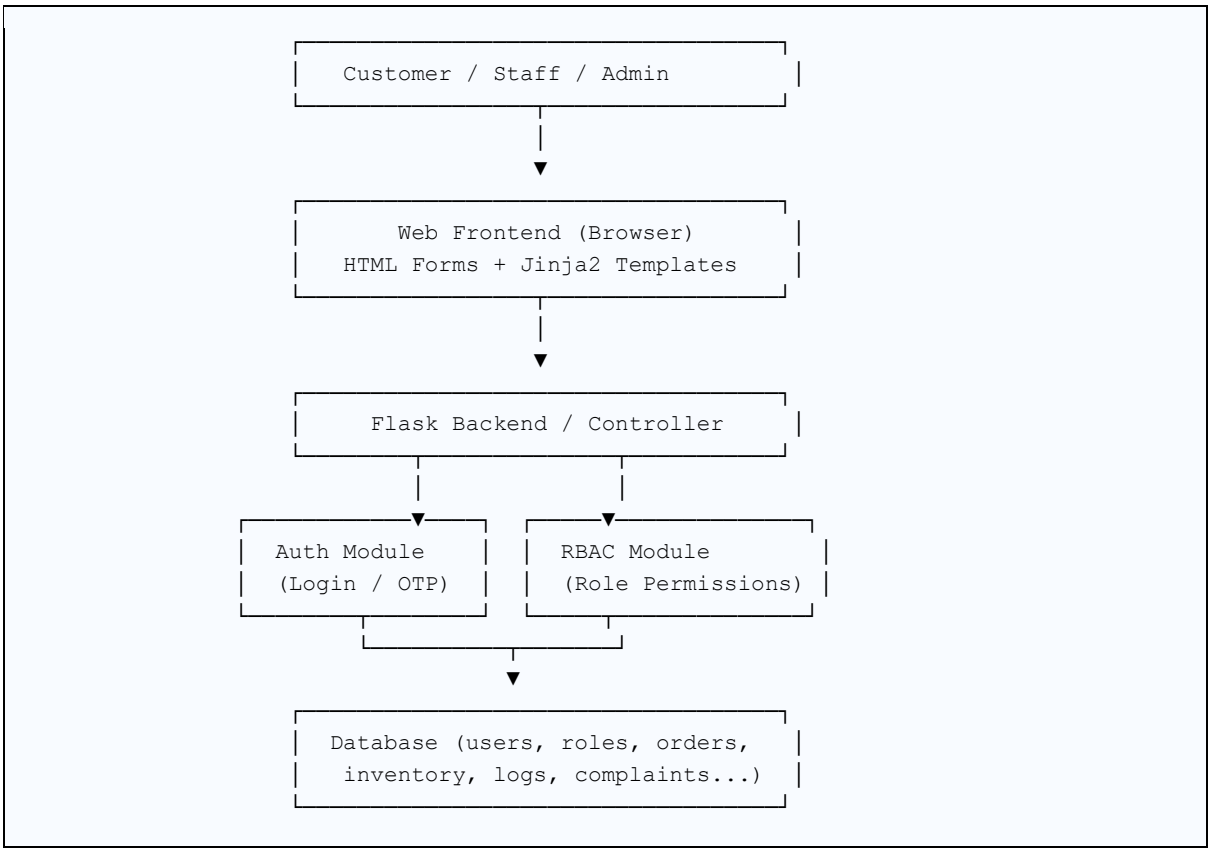


*Figure 1 — System Overview Diagram*

# 2. SYSTEM DESIGN

## 2.1. Application Design

The system is designed as a web-based Identity Security Framework for a retail platform. It follows a client-server architecture where users interact through a browser interface and all authentication, authorisation, and role management are handled at the backend.

The application supports three main user roles: Customer, Staff, and Admin. Each user interacts through a centralised login portal with two-factor authentication, and access is enforced using Role-Based Access Control (RBAC).

The backend handles credential validation, OTP generation and delivery, session management, inventory management, order processing, attendance tracking, complaint resolution, and security audit logging.

| Layer | Technology | Responsibility |
|---|---|---|
| **Model** | SQLite 3 (10 tables) | Persistent storage — users, orders, inventory, logs, salary, complaints |
| **View** | Jinja2 HTML + CSS3 | Server-side rendered dashboards, forms, tables, and printable invoices |
| **Controller** | Python 3.11 + Flask | HTTP routing, input validation, session checks, database queries |
| **Security** | Werkzeug PBKDF2-SHA256 | Password hashing, OTP generation, account lockout enforcement |
| **Email** | smtplib + Gmail SMTP | Delivers 6-digit OTP for signup, login 2FA, and password reset |

### 2.2. Process Flow

The system follows a structured identity verification and access control flow:

- **Step 1 — User Login Request:** User enters username and password through the login interface
- **Step 2 — Authentication:** System verifies credentials using PBKDF2-SHA256 hashed password comparison via Werkzeug
- **Step 3 — Lockout Check:** If failed attempts reach 5, account is locked for 10 minutes automatically
- **Step 4 — OTP Generation:** A 6-digit One-Time Password is generated and sent to the user's registered email
- **Step 5 — Role Retrieval:** User role (Customer / Staff / Admin) is fetched from the database
- **Step 6 — Authorisation (RBAC):** System checks permissions based on assigned role
- **Step 7 — Session Creation:** Permanent session keys (user, role) created; temporary OTP keys deleted
- **Step 8 — Activity Logging:** Login attempt logged to login_logs with username, IP address, timestamp, and status

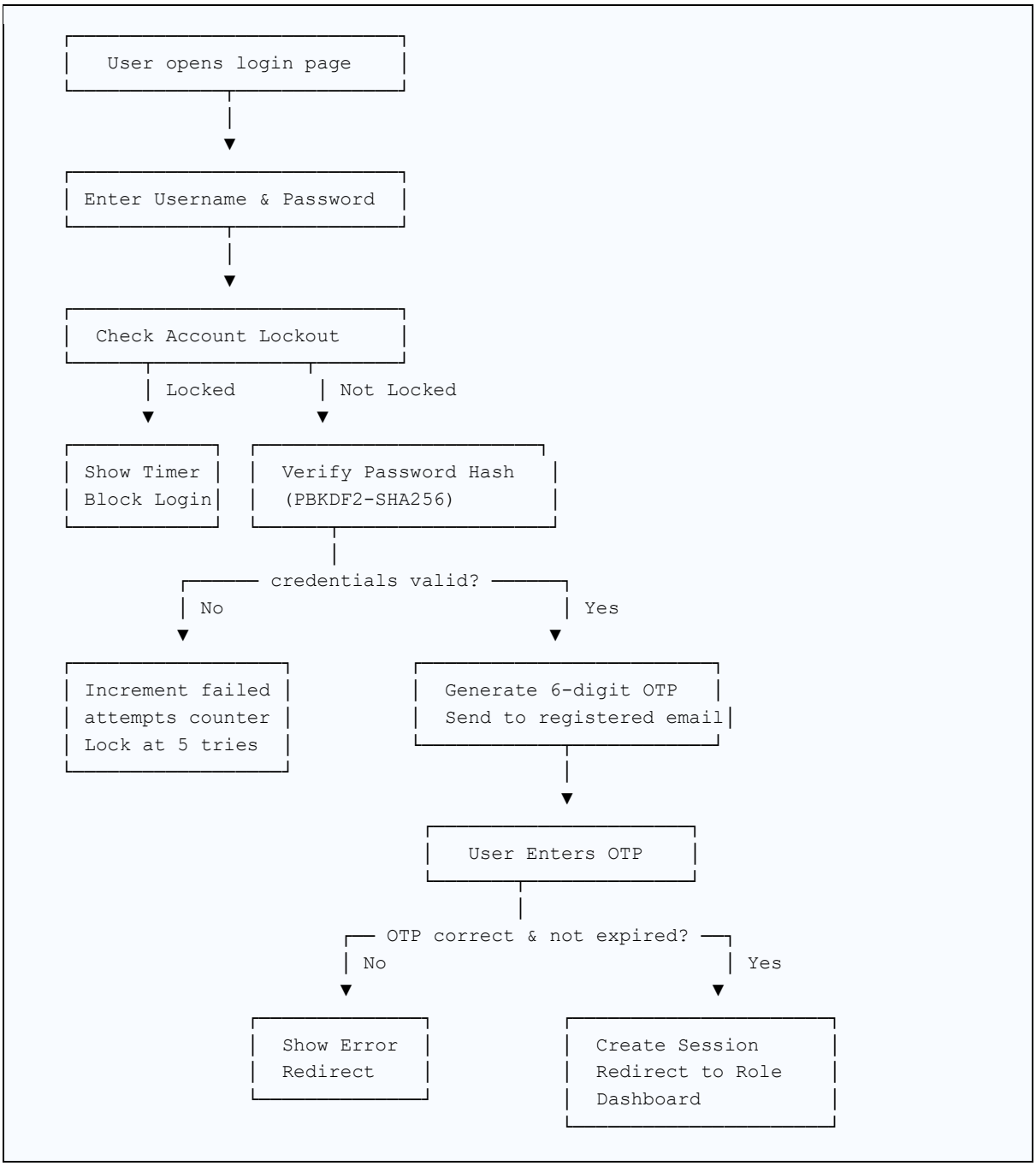


*Figure 2 — Login Process Flow Diagram*

### 2.3. Information Flow

Data flows through the system in the following direction:

- User Input (Login Credentials) → Web Frontend (Browser)
- Frontend → Flask Backend Controller (HTTP POST)
- Backend → SQLite Database (parameterised credential query)
- Database → Role and user information returned to Flask
- Flask → smtplib → Gmail SMTP → User Email Inbox (OTP delivery)
- Role → RBAC Enforcement Module (session role check on every route)
- RBAC Module → Access Decision (Allow / Deny) → Role Dashboard
- All login events → login_logs table (username, IP address, timestamp, status)

```
BROWSER          FLASK APP          SESSION STORE        DATABASE
   |                 |                    |                  |
   |— HTTP POST ——▶|                    |                  |
   |   (credentials) |— Read ————————▶|                  |
   |                 |◀— role, user ———|                  |
   |                 |— SQL Query (?) ——————————————▶|
   |                 |◀— Result rows ——————————————————--|
   |                 |— Write ————————▶|                  |
   |                 |   (OTP, temp_user)  |              |
   |◀— HTML / Redir—|                    |                  |
   |                 |— smtplib ——————————————————————▶Gmail SMTP
   |                 |   STARTTLS / port 587               |
   |                 |                                     ▼
   |                 |                              User Email Inbox
```

*Figure 3 — Information Flow Diagram*

### 2.4. Components Design

The system is composed of the following major components:

| Component | Description |
|---|---|
| **User Interface (Frontend)** | Browser-based pages — login, signup, OTP, and role-based dashboards for Customer, Staff, and Admin |
| **Authentication Module** | Handles user login with password hashing, email OTP 2FA, account lockout after 5 failed attempts, forgot and reset password with reuse prevention |
| **Password Security Module** | Stores and verifies PBKDF2-SHA256 hashed passwords via Werkzeug. Plain passwords are never stored or logged at any point |
| **Authorisation Module (RBAC)** | Enforces role-based access on every protected route. session['role'] is checked before any database query executes. Unauthorised access redirects to /login |
| **Session Management Module** | Flask signed-cookie sessions. Temporary OTP and form keys deleted after each flow. Permanent keys (user, role) persist until logout via session.clear() |
| **Staff Management Module** | Inventory (30 products, low stock alerts), Reports (daily/weekly/monthly), Working Hours (clock-in/clock-out), Salary History, Complaint Resolution |
| **Customer Module** | My Orders (weight selector 250g–7kg, 30 products), Printable Invoice, Profile, Reward Score (Gold/Silver tiers), Announcements, Complaint Box |
| **Database** | SQLite — 10 auto-created tables: users, login_logs, inventory, sales, returns, customer_orders, complaints, announcements, staff_hours, salary |
| **Logging & Monitoring Module** | Records every login attempt in login_logs with username, IP address, timestamp, and success/failure status. Visible to Admin only |

```
COMPONENT INTERACTION DIAGRAM

┌──────────────────────────────────────────────────────────────────┐
│                FLASK CONTROLLER  (app.py)                        │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐           │
│  │ AUTH MODULE  │  │    ADMIN     │  │ RBAC CONTROL │           │
│  │ signup/login │  │  user mgmt   │  │ session role │           │
│  │ OTP/lockout  │  │  audit logs  │  │ check every  │           │
│  │ pwd reset    │  │  announce    │  │ route        │           │
│  └──────┬───────┘  └──────┬───────┘  └──────┬───────┘           │
│  ┌──────▼───────┐  ┌──────▼──────────────────▼──────┐           │
│  │    STAFF     │  │          CUSTOMER MODULE       │           │
│  │  inventory   │  │  orders (weight + qty)         │           │
│  │  reports     │  │  invoice / profile / rewards   │           │
│  │  hours/salary│  │  complaints / announcements    │           │
│  └──────┬───────┘  └──────────────┬─────────────────┘           │
└─────────┼──────────────────────────┼──────────────────────────────┘
          └──────────────┬───────────┘
                         ▼
              ┌──────────────────────────┐
              │         SQLite DB        │
              │  users        login_logs │
              │  inventory    customer_orders │
              │  staff_hours  salary     │
              │  complaints   announcements │
              └──────────────────────────┘
```

*Figure 4 — Component Interaction Diagram*

### 2.5. Key Design Considerations

- Role-Based Access Control (RBAC) ensures users only access permitted resources based on their assigned role (Customer, Staff, Admin)
- Passwords are stored using PBKDF2-SHA256 hashing with a random 16-byte salt via Werkzeug — plain passwords are never written to the database or logs
- Account lockout after 5 consecutive failed login attempts for 10 minutes prevents brute-force attacks
- Email OTP two-factor authentication requires both password and a time-limited code, preventing account takeover even after credential theft
- Session management uses Flask signed cookies — tampered sessions are rejected. All temporary keys (OTP, temp_user) are deleted after each flow
- Generic login error messages prevent username enumeration — the same message is shown whether the username or password is wrong

- The principle of least privilege is enforced — users are given only the minimum access their role requires
- POST-Redirect-GET pattern is used on all form submissions to prevent double submission on browser back/refresh
- All authentication attempts are logged with IP address and timestamp for security auditing

### 2.6. API Catalogue

This system does not expose or consume external REST APIs. All data processing is internal via Flask route functions. The only external integration is Gmail SMTP for outbound OTP email delivery. The following internal function interfaces are defined:

| Function | Input | Output | Description |
|---|---|---|---|
| **authenticate_user()** | Username, Password | Success / Failure | Validates credentials using check_password_hash() and checks lockout status |
| **generate_otp()** | None | 6-digit string | Generates a random One-Time Password for email delivery |
| **otp_expired()** | Timestamp | Boolean (True/False) | Returns True if more than 5 minutes have elapsed since OTP was generated |
| **strong_password()** | Password string | Boolean (True/False) | Checks minimum 8 chars, one uppercase letter, and one digit |
| **get_user_role()** | Username | Role (Customer/Staff/Admin) | Retrieves the assigned role from the users table |
| **authorize_access()** | session['role'] | Allow / Deny | Enforces RBAC — rejects request if role does not match required level |
| **set_error() / get_error()** | Error message string | None / String | Stores error in session before redirect; reads and deletes it on next GET |
| **send_otp()** | Email, OTP | None (sends email) | Delivers HTML-formatted OTP email via Gmail SMTP with STARTTLS on port 587 |

# 3. DATA DESIGN

## 3.1. Data Model

The system uses a relational data model to store and manage identity, retail, HR, and communication data. The database consists of 10 tables created automatically by init_db() on first application run.

```
DATABASE SCHEMA — database.db  (10 tables, auto-created on first run)

┌──────────────────────────────┐  ┌──────────────────────────────┐
│   IDENTITY & AUTH            │  │   RETAIL OPERATIONS          │
│                              │  │                              │
│  users                       │  │  inventory                   │
│  ├ id (PK)                   │  │  ├ product_name, category    │
│  ├ username (UNIQUE)         │  │  ├ quantity, price           │
│  ├ email (UNIQUE)            │  │  └ low_stock_threshold       │
│  ├ password (PBKDF2 hash)    │  │                              │
│  ├ role (Admin/Staff/Cust.)  │  │  customer_orders             │
│  ├ failed_attempts           │  │  ├ username, product_name    │
│  └ lock_time (nullable)      │  │  ├ quantity, total_price     │
│                              │  │  ├ weight (250g-7kg)         │
│  login_logs                  │  │  └ purchase_type             │
│  ├ username, ip_address      │  │                              │
│  ├ login_time                │  │  sales / returns             │
│  └ status (SUCCESS/FAILED)   │  └──────────────────────────────┘
└──────────────────────────────┘
                                 ┌──────────────────────────────┐
┌──────────────────────────────┐  │   HR & COMMUNICATION         │
│   CUSTOMER SERVICE           │  │                              │
│                              │  │  staff_hours                 │
│  complaints                  │  │  ├ username, date            │
│  ├ username, subject         │  │  ├ clock_in, clock_out       │
│  ├ message                   │  │  └ hours_worked              │
│  └ status (Open/Resolved)    │  │                              │
└──────────────────────────────┘  │  salary, announcements       │
                                 └──────────────────────────────┘
```

*Figure 5 — Logical Data Model*

| Table | Area | Key Columns & Purpose |
|---|---|---|
| **users** | Identity | id, username (UNIQUE), email (UNIQUE), password (hash), role, failed_attempts, lock_time |
| **login_logs** | Audit | id, username, ip_address, login_time, status (SUCCESS / FAILED) |
| **inventory** | Operations | id, product_name, category, quantity, price, low_stock_threshold |
| **sales** | Operations | id, product_id, product_name, quantity, total_price, staff_username, sale_time |
| **returns** | Operations | id, product_name, quantity, reason, staff_username, return_time, status |
| **customer_orders** | Customer | id, username, product_name, quantity, total_price, order_time, weight, purchase_type |
| **complaints** | Customer | id, username, subject, message, status (Open / Resolved), submitted_time |
| **announcements** | Comms | id, title, message, posted_by, posted_time |
| **staff_hours** | HR | id, username, date, clock_in, clock_out, hours_worked |
| **salary** | HR | id, username, month, amount, status (Paid / Pending), paid_date |

### 3.2. Data Access Mechanism

- The system uses SQLite via Python's sqlite3 module for all database operations
- All authentication requests use parameterised ? placeholder queries to prevent SQL injection
- Role information is fetched from the database after successful authentication to determine access level
- Session data is stored in a signed Flask cookie and validated on every request
- Access to sensitive data is restricted based on RBAC rules enforced at the route level

### 3.3. Data Retention Policies

- User Credentials: Stored permanently in the database in PBKDF2-SHA256 hashed format
- Session Data: Stored only for active sessions — cleared on logout via session.clear()
- Activity Logs: Login logs retained for auditing. In production, logs older than 90 days would be purged
- Customer Orders: Retained for the lifetime of the database. Production: 12-month archive policy
- Salary Records: Retained indefinitely. Production: minimum 7 years for financial compliance
- Data Privacy: Sensitive information is never exposed to unauthorised roles or logged in plain form

### 3.4. Data Migration

Data migration is not applicable in the current academic context. The database schema is fixed and init_db() handles all table creation and seeding automatically on first run. If the schema requires updating, the database.db file can be deleted and recreated. For production deployment, Flask-Migrate (Alembic) would manage versioned schema migrations.

## 4. INTERFACES

The system provides a web-based interface accessible through a browser at http://localhost:5000. The interface enables secure interaction between users and the retail identity security system, supporting two-factor authentication, role-based access, inventory management, order processing, and user management.

| UI Element | Type | Description |
|---|---|---|
| **Login Page** | HTML Form | Split-card layout — left: Retail Shoppeee brand and trust message; right: username/password form with Create Account and Forgot Password links |
| **OTP Verification Page** | HTML Form | Shared page for signup, login 2FA, and password reset. Displays 6-digit OTP input with 5-minute expiry warning |
| **Create Account Page** | HTML Form | Registration form with username, email, password (strength enforced), and role selector (Customer / Staff) |
| **Customer Dashboard** | Dashboard | Live stats: orders, total spent, reward points, complaints, announcements. Reward score bar. 5 module navigation cards |

| | | |
|---|---|---|
| **My Orders Page** | Form + Table | Product dropdown (30 items), weight selector (250g–7kg), quantity input, Place Order button, order history table with invoice links |
| **Invoice Page** | Printable View | Printable bill with invoice number, product, weight, quantity, unit price, order date, purchase type, status, and total amount |
| **Staff Dashboard** | Dashboard | Live stats: products in stock, low stock alerts, open complaints, announcements. 7 module navigation cards |
| **Inventory Page** | Admin Interface | Add product form (name, category, qty, price, threshold). Full stock table with In Stock / Low Stock / Out of Stock badges |
| **Working Hours Page** | Form + Log | Clock In / Clock Out buttons with daily attendance log showing hours worked per day |
| **Admin Security Dashboard** | Admin Interface | User list with role badges, unlock/delete actions. Login activity log with username, IP, timestamp, and status |
| **Error Messages** | UI Feedback | Error alerts displayed after redirect using set_error()/get_error() — shown once, cleared on refresh |
| **Success Notifications** | UI Feedback | Success alerts confirming actions such as order placed, complaint submitted, password updated |

# 5. STATE AND SESSION MANAGEMENT

The system uses session-based state management via Flask's signed-cookie mechanism to maintain user authentication and control access across all parts of the application. Since the application follows a request-response model, session handling is essential to track logged-in users across multiple pages.

State is managed using server-side session mechanisms provided by the Flask framework, signed with HMAC-SHA1 using the application secret key. The following session keys are used to maintain state during user interaction:

- user: Identifies the currently logged-in user (permanent until logout)
- role: Determines access permissions — Customer, Staff, or Admin (permanent until logout)

- temp_user / temp_role: Temporary keys during two-factor login — deleted after OTP is confirmed
- signup_otp / signup_time / signup_data: Temporary keys during registration — deleted after OTP verified
- otp / otp_time / reset_email: Temporary keys during password reset flow
- error: Error message passed between pages via set_error() / get_error() — read once and deleted

Key session management features include:

- Session Creation: Created after both password and OTP are successfully verified
- Session Validation: session['role'] is checked at the start of every protected route
- Role-Based State Control: User role stored in session is used for RBAC enforcement on every request
- Session Termination: session.clear() destroys all session data on logout
- Temporary State Cleanup: All OTP and temp keys are explicitly deleted after each multi-step flow

# 6. CACHING

The following caching approach is applied in the current system:

- No server-side caching is implemented in the current academic version — each page is rendered fresh on every request
- Static resources such as style.css are cached by the browser automatically to reduce load times
- Sensitive data including passwords, session tokens, and OTPs are never cached at any level
- Session-related data is managed using Flask's secure signed-cookie mechanism, not browser caches
- Database queries are parameterised and executed per request — no query result caching is applied

In a production deployment, Redis-backed Flask-Caching with a TTL of 60–300 seconds would be used for inventory listings and announcements, which change infrequently. This controlled caching strategy would balance system performance with data freshness and security.

# 7. NON-FUNCTIONAL REQUIREMENTS

## 7.1. Security Aspects

- User passwords are stored using PBKDF2-SHA256 hashing with a 16-byte random salt — plain passwords are never stored or logged
- Role-Based Access Control (RBAC) ensures users only access resources permitted to their assigned role
- Email OTP two-factor authentication is required after every password verification — the account cannot be accessed with a password alone
- Account lockout after 5 consecutive failed attempts for 10 minutes prevents brute-force and credential stuffing attacks
- Generic login error messages prevent username enumeration — attacker cannot determine which usernames are registered
- Session management prevents session hijacking — sessions are signed, temporary keys are deleted after use, and session.clear() is called on logout
- Input validation is applied to all user inputs — password strength enforced server-side
- All login attempts are logged with username, IP address, and timestamp for security monitoring and incident response
- The principle of least privilege is enforced — each role receives only the minimum access required
- STARTTLS encryption is used for all SMTP connections to protect OTP codes in transit

## 7.2. Performance Aspects

- Page response time < 100 ms for all local requests — server-side rendering with no external JavaScript or CDN dependencies
- OTP email delivery takes 1–3 seconds due to Gmail SMTP connection time — acceptable for academic demonstration
- All primary user lookups use UNIQUE indexed columns (username, email) ensuring fast database query performance
- Application startup completes in under 2 seconds — init_db() skips seeding if tables already contain data
- Lightweight backend design with no heavy frameworks minimises server load and supports multiple concurrent browser sessions
- Static resource caching (style.css) at the browser level reduces page load time on repeat visits

# 8. REFERENCE


- Flask Documentation — https://flask.palletsprojects.com/
- Werkzeug Security Utilities — https://werkzeug.palletsprojects.com/en/latest/utils/
- OWASP Authentication Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OWASP Session Management Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- Python sqlite3 Documentation — https://docs.python.org/3/library/sqlite3.html
- Python smtplib Documentation — https://docs.python.org/3/library/smtplib.html
- NIST SP 800-63B Digital Identity Guidelines — https://pages.nist.gov/800-63-3/sp800-63b.html
- Role-Based Access Control (RBAC) Concepts — https://csrc.nist.gov/projects/role-based-access-control
- Web Application Security Best Practices — https://owasp.org/